

SQL Query Performance Analytics Report

Prepared By

Name: Gayatri kasbekar

Tools Used: Python, MySQL, Power BI

Connect With Me :-

[github](#)

[linkedin](#)

Index

1. Project Overview
2. Problem Statement
3. Objectives
4. Tools & Technologies Used
5. Dataset Description
6. Data Cleaning Process
7. Exploratory Data Analysis (EDA)
8. SQL Database Design & Analysis
9. Power BI Dashboard Development
10. Key Performance Indicators (KPIs)
11. Business Recommendations
12. Conclusion
13. Future Improvements

1. Project Overview

This project focuses on analyzing database query performance using Python, MySQL, and Power BI. The primary goal of the project is to monitor query execution patterns, analyze system resource utilization, identify failed query executions, and create an interactive dashboard for performance monitoring.

The project follows a complete end-to-end analytics workflow including:

- Data Cleaning
- Exploratory Data Analysis (EDA)
- SQL Analysis
- Dashboard Development
- Business Insight Generation

The dashboard provides insights into:

- Query execution trends
- Query success and failure rates
- Department-wise database activity
- CPU and memory utilization
- Query execution performance

2. Problem Statement

Organizations generate large volumes of database query logs every day. Monitoring these logs is important for maintaining database performance, identifying system bottlenecks, reducing failed executions, and improving operational efficiency.

However, raw query log data often contains:

- Missing values
- Duplicate records
- Inconsistent formatting
- Invalid entries

- Resource usage anomalies

This project aims to clean and analyze query execution data to derive meaningful business and technical insights.

3. Objectives

The main objectives of this project are:

- Clean and preprocess raw query log data.
- Analyze query execution performance.
- Monitor CPU and memory usage.
- Identify failed and timeout queries.
- Analyze department-wise query activity.
- Build an interactive Power BI dashboard.
- Generate business insights and recommendations.

4. Tools & Technologies Used

Tool / Technology	Purpose
Python	Data Cleaning & Analysis
Pandas	Data Manipulation
NumPy	Numerical Computation
Matplotlib	Data Visualization
Seaborn	Statistical Visualization
MySQL	SQL Database Analysis

Power BI

Dashboard Development

VS Code / Jupyter
Notebook

Development Environment

5. Dataset Description

The project uses two datasets:

A. Users Dataset

This dataset contains user information.

Column Name	Description
user_id	Unique identifier for users
username	Name of the user
department	Department associated with user

Data Issues Found

- Missing department values
- Duplicate usernames
- Inconsistent department naming
- Leading and trailing whitespaces
- Mixed casing issues

B. Query Logs Dataset

This dataset contains query execution details.

Column Name	Description
log_id	Unique query log identifier

user_id	User identifier
query_type	Type of SQL query
execution_time	Query execution duration
cpu_usage	CPU utilization
memory_usage	Memory consumption
execution_date	Query execution date
status	Query execution status

Data Issues Found

- Missing user IDs
- Missing memory usage values
- Invalid execution dates
- Negative execution times
- Duplicate records
- Inconsistent query type values

6. Data Cleaning Process

Data cleaning was performed using Python and Pandas.

Steps Performed

6.1 Handling Missing Values

- Removed rows with missing critical identifiers.
- Filled or handled missing department values.
- Removed invalid user IDs.

6.2 Removing Duplicates

- Removed duplicate username and department combinations.

- Removed duplicate query records.

6.3 Standardizing Text Data

Performed:

- Lowercase conversion
- Whitespace removal
- Standardization of department names
- Standardization of query types

Example:

Original Value	Standardized Value
HR	hr
Human Resources	hr
ENGINEERING	engineering
select	SELECT

6.4 Data Type Conversion

Column	Converted Type
execution_date	datetime
execution_time	float
cpu_usage	float
memory_usage	float

6.5 Outlier Handling

Outliers in execution_time were identified using the IQR (Interquartile Range) method.

Formula Used

- Lower Bound = $Q1 - 1.5 \times IQR$
- Upper Bound = $Q3 + 1.5 \times IQR$

Extreme execution times were removed to improve analysis accuracy.

6.6 Data Validation

Validated:

- Foreign key consistency between logs and users tables
- Valid user IDs
- Valid execution dates

7. Exploratory Data Analysis (EDA)

EDA was performed using:

- Pandas
- NumPy
- Matplotlib
- Seaborn

7.1 KPI Analysis

KPIs Generated

KPI	Value
Total Queries	1601
Active Users	1097
Average Execution Time	1.12 sec
Average CPU Usage	15.79%
Average Memory Usage	29.43 MB
Success Rate	58.78%

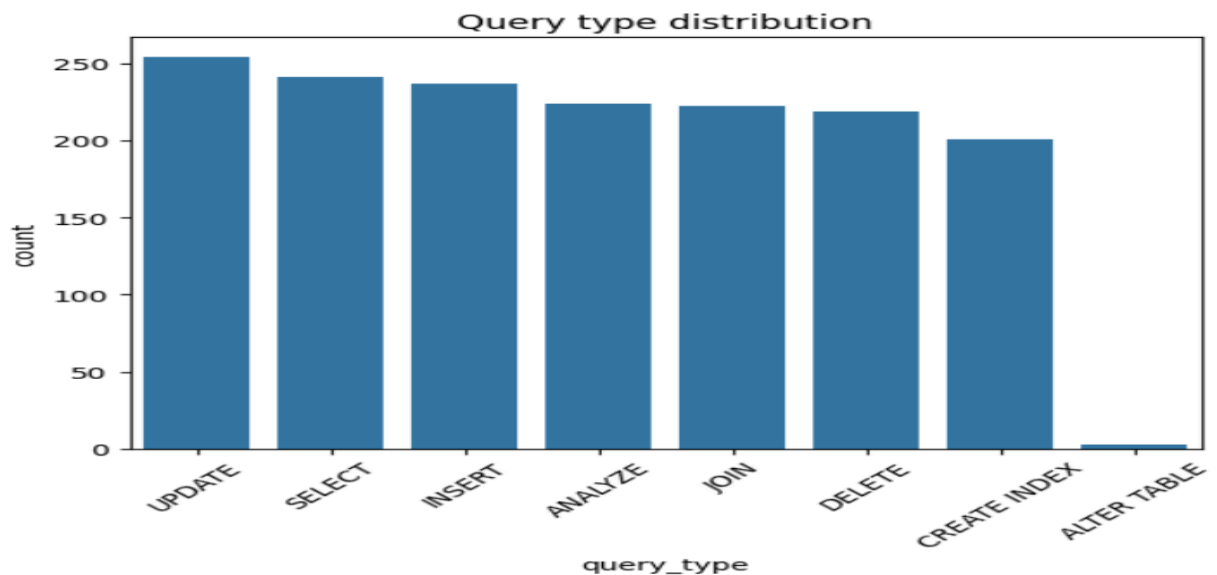
Failed Queries	336
Most Used Query Type	UPDATE
Most Active Department	IT Support

7.2 Query Type Distribution

A bar chart was used to analyze the distribution of query types.

Observations

- UPDATE queries were executed most frequently.
- SELECT and INSERT queries were also heavily used.
- ALTER TABLE queries were least frequent.



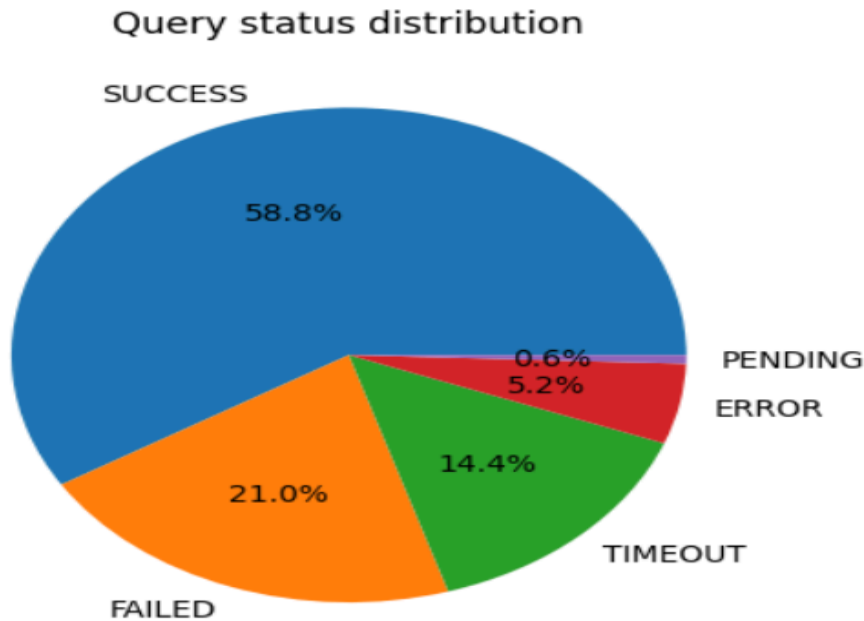
7.3 Query Status Distribution

A donut chart was used to analyze query execution status.

Observations

- Majority of queries executed successfully.

- A significant number of FAILED and TIMEOUT queries were observed.
- ERROR and PENDING queries were relatively low.

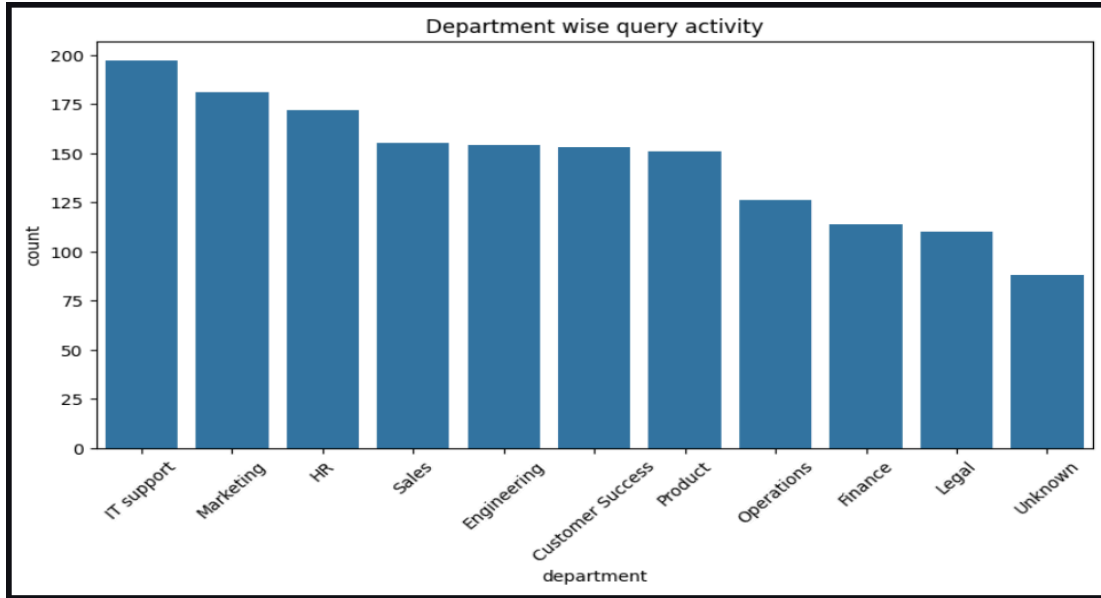


7.4 Department-wise Query Activity

A bar chart was used to analyze database activity across departments.

Observations

- IT Support generated the highest query volume.
- Marketing and HR also showed high activity.
- Finance and Legal departments generated comparatively lower workloads.

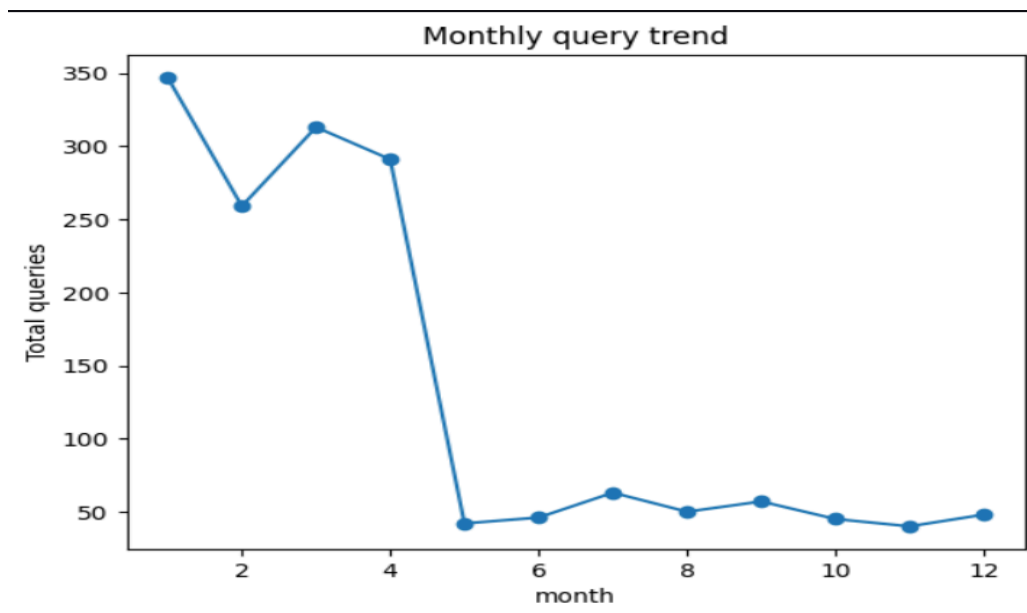


7.5 Monthly Query Trend

A line chart was used to analyze monthly query activity.

Observations

- Query activity was highest during the initial months.
- Query volume declined gradually later in the year.
- The trend indicates workload fluctuation over time.

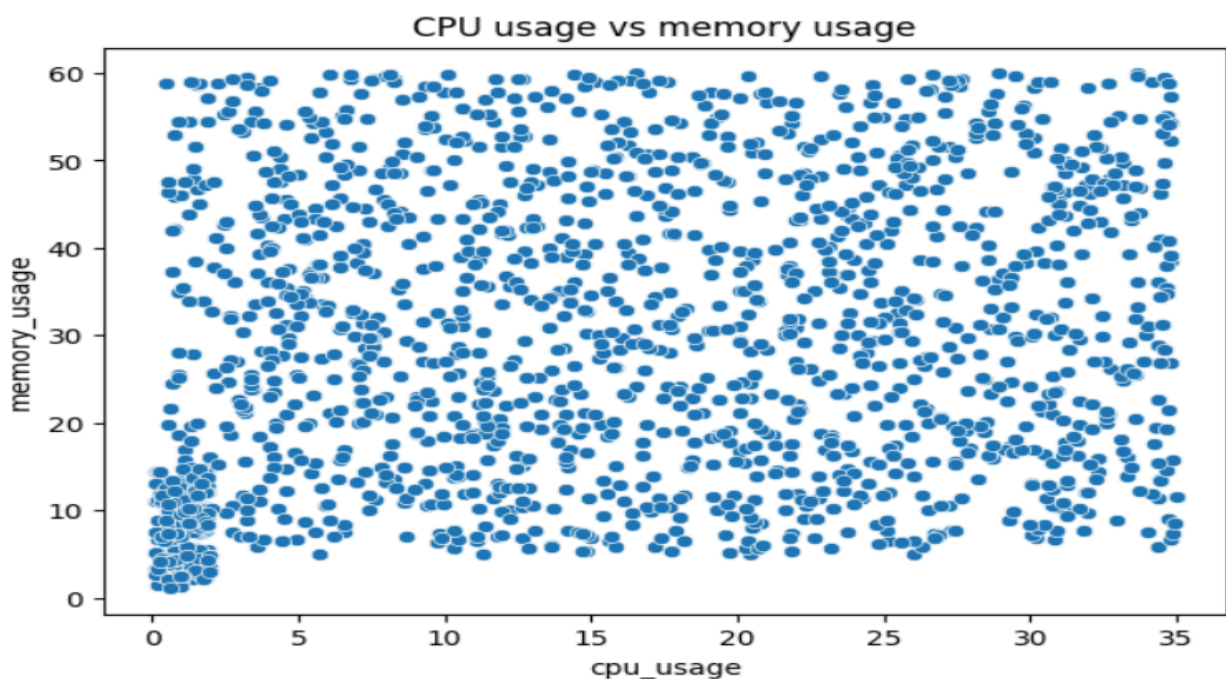


7.6 CPU vs Memory Usage

A scatter plot was used to study the relationship between CPU and memory usage.

Observations

- CPU and memory usage did not show strong linear correlation.
- Resource utilization remained within acceptable ranges.
- Different query types consumed resources differently.



8. SQL Database Design & Analysis

8.1 Database Schema

Users Table

```
create table users(  
  user_id int primary key,  
  username varchar(50),  
  department varchar(50)  
);
```

Logs Table

```
create table Logs_(  
log_id int primary key,  
user_id int,  
query_type varchar(20),  
execution_time float,  
cpu_usage float,  
memory_usage float,  
execution_date date,  
status varchar(50),  
foreign key(user_id) references users(user_id)  
);
```

8.2 SQL Analysis Queries

Total Queries

```
select count(*) as total_queries  
from logs_;
```

Department-wise Query Count

```
select department,count(*)as total_queries  
from logs_ l join users u  
on l.user_id=u.user_id  
group by department  
order by total_queries desc;
```

Success Rate

```
select  
round(  
sum(case when status='SUCCESS' then 1 else 0 end)*100.0/count(*),2) as  
success_rate  
from logs_;
```

Most Active Users

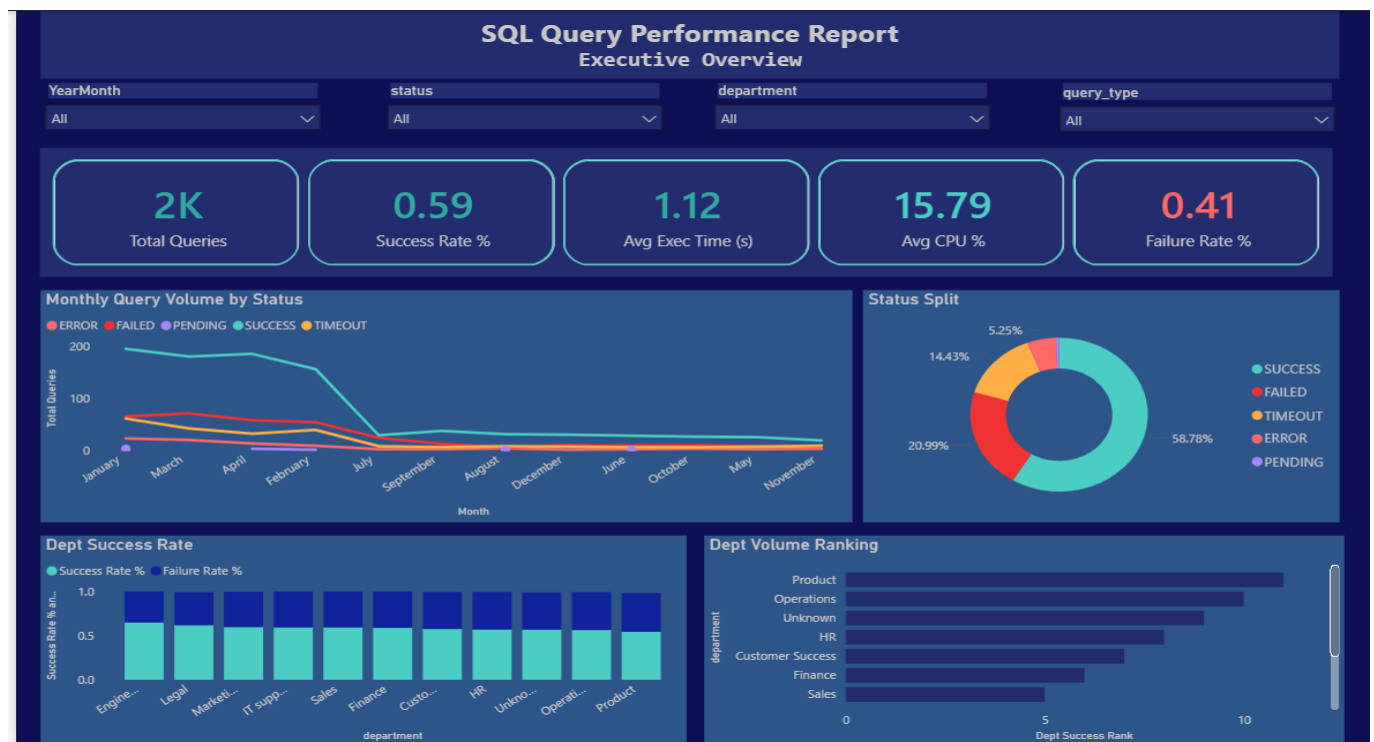
```
select username,count(*) as total_queries  
from logs_ l  
join users u  
on l.user_id=u.user_id  
group by username  
order by total_queries desc
```

limit 10;

9. Power BI Dashboard Development

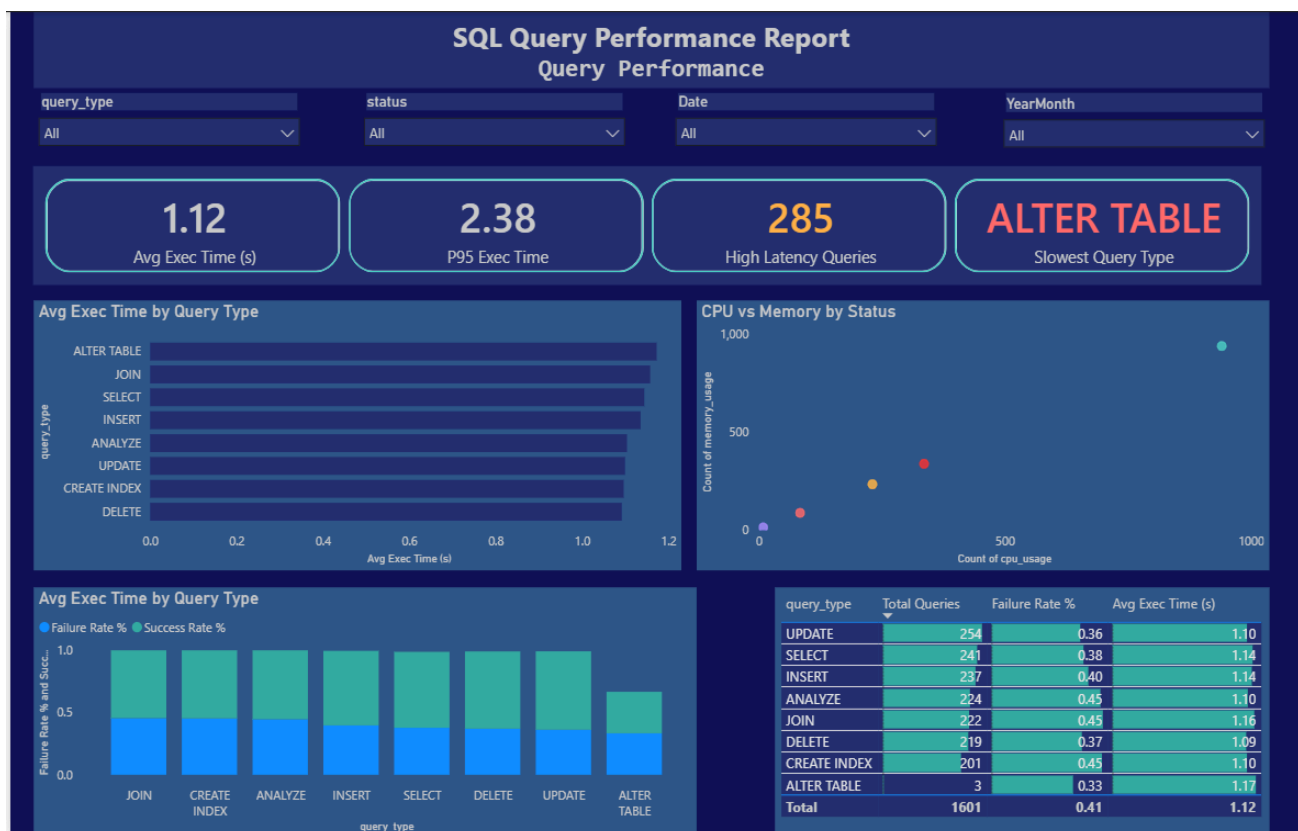
9.1 Executive Overview Dashboard

1. This dashboard provides an overall summary of SQL query performance and system activity.
2. It displays important KPIs such as total queries, success rate, failure rate, execution time, and CPU usage.
3. The dashboard shows that 59% of queries were successful, while 41% failed, indicating system reliability issues.
4. Monthly query trends help identify workload patterns and periods of high database activity.
5. Department-wise analysis shows that Product and Operations teams generate the highest query volume.
6. The dashboard helps management monitor database health and make performance-related decisions quickly.



9.2 Query Performance Dashboard

- This dashboard focuses on detailed technical analysis of SQL query performance.
- It highlights important metrics such as average execution time, P95 latency, and high-latency queries.
- Around 285 high-latency queries were detected, showing major performance bottlenecks.
- Query types like ALTER TABLE, JOIN, and SELECT consume the highest execution time and resources.
- CPU and memory usage analysis helps identify resource-intensive query operations.
- The dashboard helps developers and database administrators optimize slow queries and improve system efficiency.



10. Key Insights & Findings

- The dashboard shows a high 41% query failure rate, indicating reliability issues.
- Around 285 high-latency queries are slowing down database performance.
- Query types such as ALTER TABLE, JOIN, and SELECT consume the most resources.
- CPU and memory usage increase together during heavy query execution.
- Product and Operations departments generate the highest query workload.

11. Business Recommendations

Based on the analysis, the following recommendations are suggested:

- Optimize slow queries using indexing and query tuning techniques.
- Reduce failed queries by identifying timeout and execution issues.
- Schedule heavy operations like ALTER TABLE during non-peak hours.
- Implement automated monitoring and alert systems for performance tracking.
- Regularly analyze query execution patterns to improve efficiency.

12. Conclusion

This project successfully analyzed database query performance using Python, MySQL, and Power BI. The project demonstrated the complete analytics workflow including data cleaning, exploratory analysis, SQL querying, and dashboard development.

- The system handles a large volume of SQL queries efficiently overall.
- High failure rates and slow queries are the main performance challenges.

- Resource-heavy queries impact CPU, memory, and execution speed.
- Query optimization can significantly improve database stability and response time.
- The dashboard helps in making data-driven decisions for database performance management.

The project highlights the importance of data-driven monitoring for improving database efficiency and operational reliability.

13. Future Improvements

- Add real-time SQL query monitoring dashboards.
- Implement automated slow-query detection and alerts.
- Include query execution plan analysis for deeper optimization.
- Add predictive analytics to identify future performance issues early.
- Enable drill-through analysis for detailed query-level investigation.